

UT2.- LENGUAJE PHP. INSERCIÓN DE CÓDIGO

EN PÁGINAS WEB

RESULTADOS DE APRENDIZAJE ASOCIADOS
2. Escribe sentencias ejecutables por un servidor Web reconociendo y aplicando procedimientos de integración del código en lenguajes de marcas.
CRITERIOS DE EVALUACIÓN
a) Se han reconocido los mecanismos de generación de páginas Web a partir de lenguajes de marcas con código embebido. b) Se han identificado las principales tecnologías asociadas. c) Se han utilizado etiquetas para la inclusión de código en el lenguaje de marcas. d) Se ha reconocido la sintaxis del lenguaje de programación que se ha de utilizar. e) Se han escrito sentencias simples y se han comprobado sus efectos en el documento resultante. f) Se han utilizado directivas para modificar el comportamiento predeterminado. g) Se han utilizado los distintos tipos de variables y operadores disponibles en el lenguaje. h) Se han identificado los ámbitos de utilización de las variables. i) Se han diseñado aplicaciones web sencillas.

UT2.- LENGUAJE PHP. INSERCIÓN DE CÓDIGO

EN PÁGINAS WEB

Índice de contenido

1.- Estructura de los contenidos.....	3
2.- PHP y HTML. Código incrustado.....	3
2.1.- Cómo ejecutar el código para visualizarlo en el navegador	5
2.2.- Sintaxis de PHP	9
2.3.- Variables y tipos de dato.....	10
2.3.1.- Declaración de variables	10
2.3.2.- Asignación por copia y por referencia	12
2.3.3.- Variables no inicializadas	13
2.3.4.- Constantes	15
2.3.5.- Tipos de datos escalares	16
2.3.6.- Ámbito de las variables	20
2.3.7.- Variables predefinidas	21
2.4.- Comentarios	23
2.5.- Expresiones y operadores en PHP	23
2.5.1.- Expresiones.....	23
2.5.2.- Operadores.....	24
2.5.3.- Orden de prioridad.....	30
2.6.- Funciones nativas de PHP	31
2.6.1.- La más utilizadas (Resumen).....	31
3.- Glosario de términos	33
4.- ANEXOS – AMPLIACIÓN DE FUNCIONES NATIVAS PHP	33
4.1.- Numéricas.....	34
4.2.- Cadenas.....	36
4.3.- De tipos.....	48
4.4.- Fechas	54
5.- Referencias bibliográficas	59

1.- Estructura de los contenidos

En este tema de introducción al lenguaje PHP, conoceremos la sintaxis básica, entenderemos cómo se integran PHP y HTML, además de describir los tipos de datos existentes y conocer algunas de las funciones más usadas en este lenguaje.

2.- PHP y HTML. Código incrustado

PHP (acrónimo recursivo de *PHP: Hypertext Preprocessor*) es un lenguaje de programación para desarrollo web en el lado servidor. Desde su aparición en 1994 ha tenido gran aceptación y se puede decir que es uno de los lenguajes más utilizados.

El lenguaje PHP es flexible y permite programar pequeños *scripts* con rapidez.

Comparado con lenguajes como Java, requiere escribir menos código y, en general, resulta menos engorroso.

Las páginas de PHP contienen HTML con código incrustado que hace "algo".

El código de PHP está encerrado entre las etiquetas especiales de comienzo y final **<?php** y **?>** que permiten entrar y salir del "**modo PHP**".



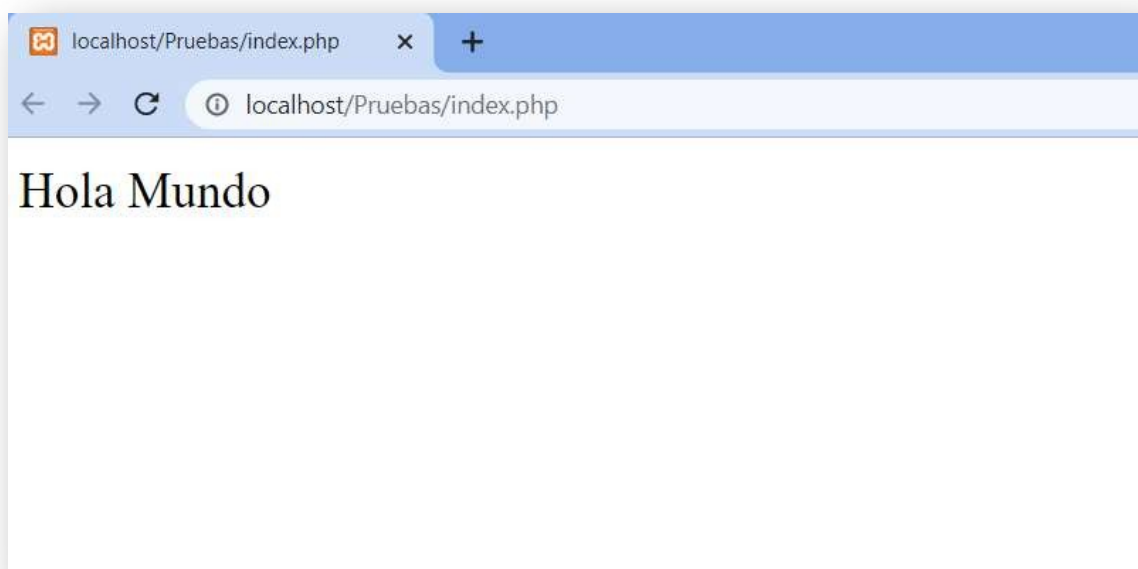
Para ejecutar un programa en php necesitamos guardarlo dentro de un servidor web.

A continuación, se muestra un ejemplo: **hola_mundo.php** que muestra una página HTML completa con un bloque PHP incrustado en las líneas 7-9 (ambas incluidas). El bloque tiene una única sentencia que sirve para mostrar la cadena "Hola mundo":

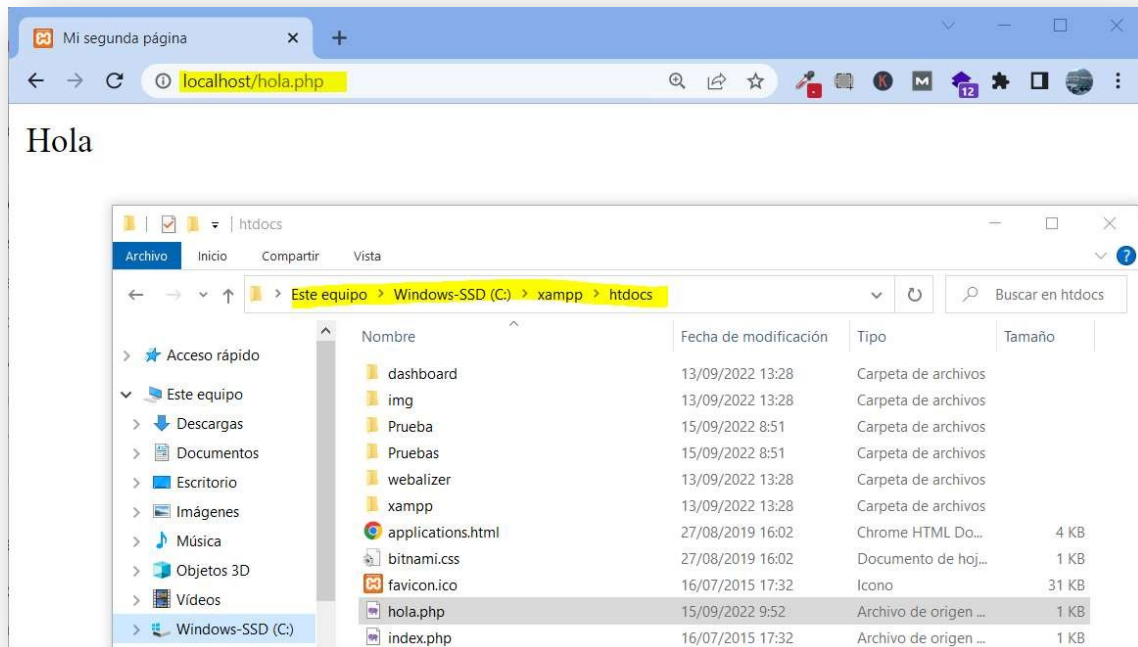


```
2 <html>
3   <head>
4       <meta charset="UTF-8">
5       <title></title>
6   </head>
7   <body>
8       <?php
9           echo "Hola Mundo"
10      ?>
11   </body>
12 </html>
13
```

Al solicitar la página al servidor web, el resultado es:

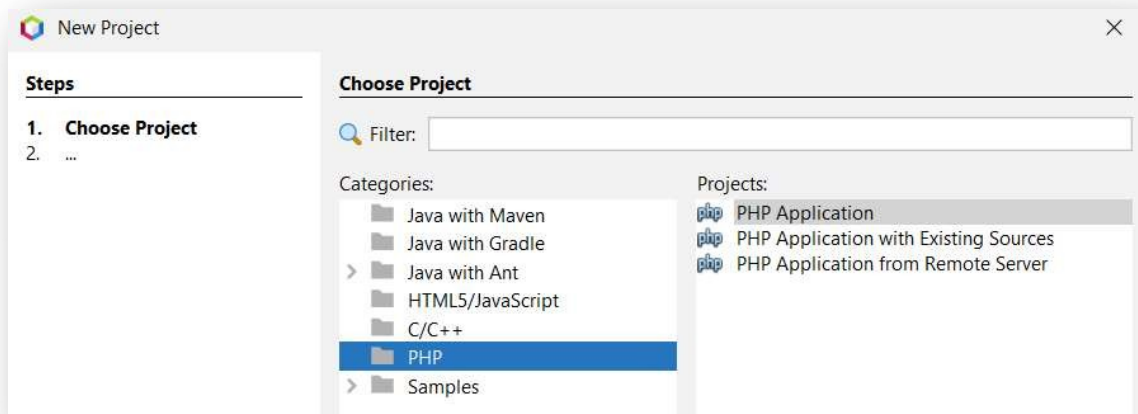


Por ejemplo, creamos **hola.php**, con el código anterior, para probar la ejecución pondré: <http://localhost/hola.php/> en el navegador.

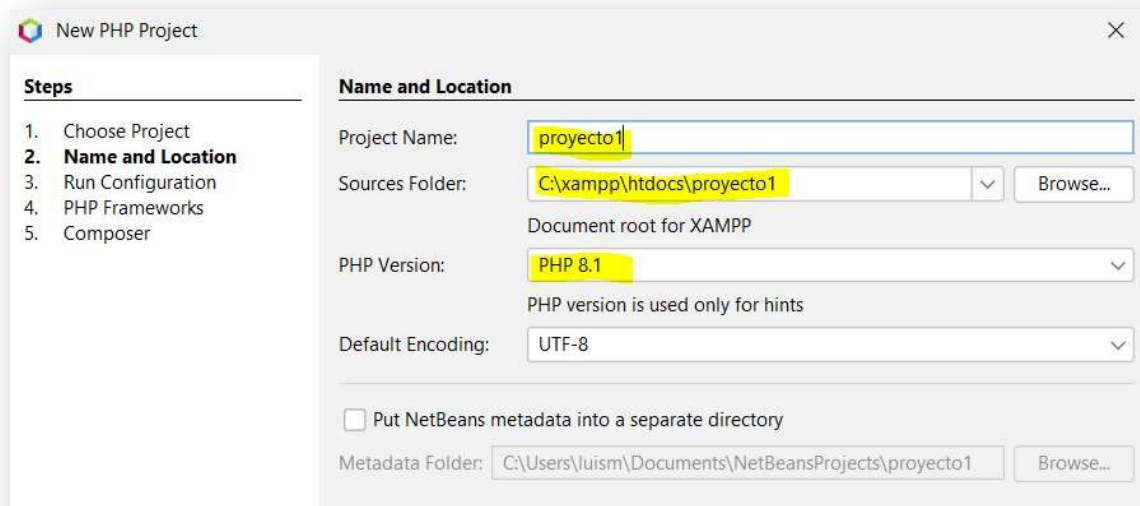


2) Utilizando Netbeans

Creamos un proyecto nuevo con *Netbeans*. **Menú File/New Project**, se selecciona **PHP/PHP Application**. Se pulsa **Next**.



Se añade el nombre del proyecto, la ubicación y se elige la codificación. En el ejemplo añadimos la ubicación dentro de la carpeta htdocs de xampp (antes creamos la carpeta proyecto1 en **c:/xampp/htdocs/proyecto1**). Ponemos un nombre al proyecto, por ejemplo, proyecto1:



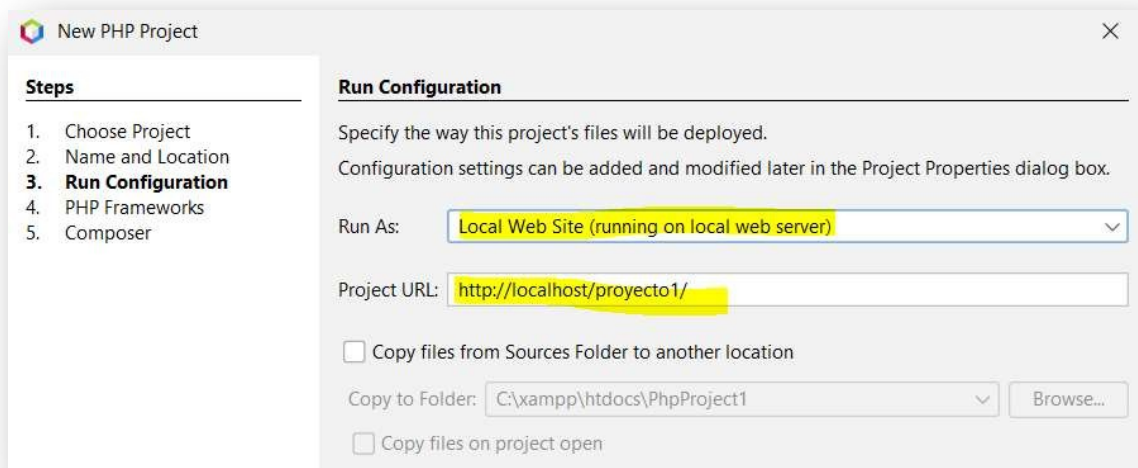
En el siguiente paso va a pedir la configuración para ejecutarse.

Indicaremos desde donde se va a ejecutar. Podemos seleccionar que se ejecute desde el servidor apache instalado en el XAMPP. En este caso el proyecto deberá estar guardado en la carpeta **c:/xampp/htdocs/**. Y en la URL del proyecto ponemos <http://localhost/proyecto1> el nombre de la carpeta donde se encuentra lo que se va a ejecutar.

Por ejemplo:

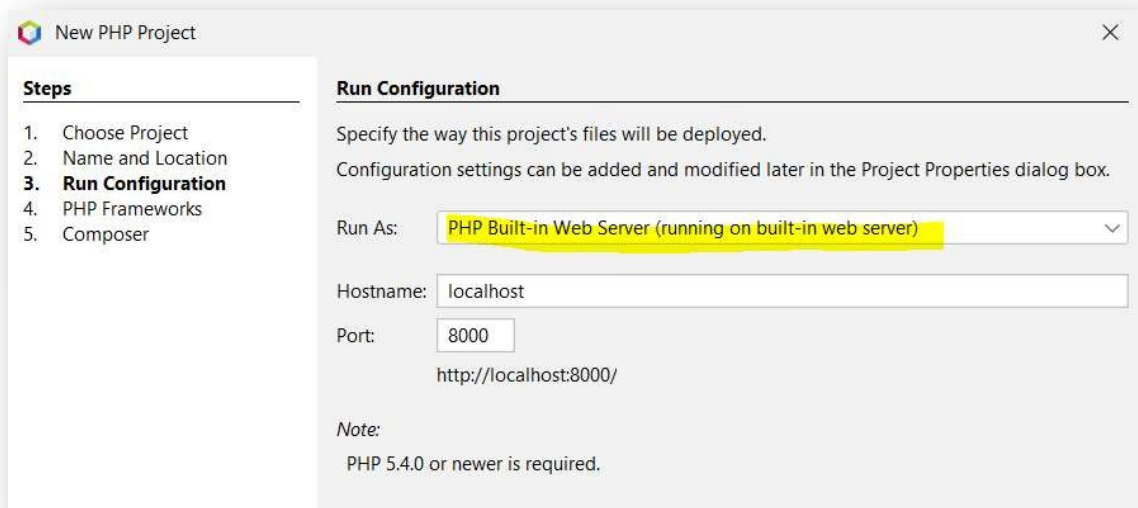
UT2.- Lenguaje PHP. Inserción de código en páginas web.

Desarrollo web en entorno servidor (2ªDAW)



También podemos indicar que se ejecute a través del *server* que proporciona NetBeans para PHP, en este caso dará igual la ubicación del proyecto. Y para ejecutar el proyecto se abrirá la URL <http://localhost:8000>.

El proyecto se ejecutará en el *hostname* que digamos y el puerto que elijamos:



¡ATENCIÓN!



Por defecto, Netbeans, nos crea el proyecto con un index.php (**o no**).

Todos los proyectos deberán tener un index.php, que es la página la cual se inicia por defecto al poner el nombre de la URL del proyecto en el navegador. <http://localhost/proyecto1/>

Si las páginas tienen otro nombre, para que se ejecuten pondremos <http://localhost/proyecto1/nombrepagina.php>

2.2.- Sintaxis de PHP

El **elemento básico** en PHP es el **bloque**. Un bloque PHP está delimitado por las etiquetas correspondientes y contiene una serie de sentencias separadas por **punto y coma**.

```
<?php
    sentencial;
    sentencian;
? >
```



Cuando un fichero contiene solo PHP se recomienda no cerrar la etiqueta del último bloque. Puede dar problemas si la respuesta del servidor involucra varios ficheros.

Referencia: <https://www.php.net/manual/es/language.basic-syntax.instruction-separation.php>

2.3.- Variables y tipos de dato

Una de las características principales de PHP es que es un lenguaje no fuertemente tipado. Esto quiere decir que **no es necesario indicar el tipo de dato al declarar una variable**.

De hecho, las variables no se declaran, se crean la primera vez que se les asigna un valor. **El tipo de dato depende del valor con que se inicialicen.**

Esto agiliza la escritura de programas, pero también tiene inconvenientes. Si no se presta atención, puede dar lugar a código de baja calidad y, a medida que las aplicaciones crecen, pueden darse errores difíciles de depurar.

2.3.1.- Declaración de variables



- Manual:

<https://www.php.net/manual/es/language.variables.php>

En PHP los identificadores de las variables van siempre precedidos por el carácter \$.

El identificador de la variable debe comenzar por:

- Una letra
- Un guión bajo ('_')

Dicho identificador puede estar formado por números, letras y guiones bajos.

Para declarar una variable, sólo hay que asignarle un valor:

\$nombre = valor;

Por ejemplo, esta sentencia declara la variable \$entero, que será de tipo *integer* porque se inicializa con un entero

\$entero = 4;



También es posible cambiar el tipo de dato de una variable simplemente asignándole un valor de otro tipo de dato. Veamos un ejemplo:

```
hola.php x
1  <?php
2  /*Declaración de variables*/
3  $entero=4; //tipo integer
4  $numero=4.5; // tipo coma flotante
5  $cadena="cadena"; //tipo cadena de caracteres
6  $bool = TRUE; //tipo booleano
7
8
9  /*Cambio de tipo de una variable*/
10 $a=5; // tipo entero
11 echo gettype($a); //imprime el tipo de dato de "a"
12 echo "<br>";
13 $a="Hola"; // El tipo de dato cambia a cadena
14 echo gettype($a); //comprobamos que ha cambiado el tipo de datos
```

La salida de este código en el navegador será la siguiente:

localhost/hola.php

integer
string

2.3.2.- Asignación por copia y por referencia



Manual:

<https://www.php.net/manual/es/language.operators.assignment.php>

En principio, la **asignación de variables** se realiza mediante **copia**.

Si hacemos:

\$a = \$b

Se crea una nueva variable *a* y se le asigna el valor que tenga *b*. Las variables *a* y *b* representan posiciones diferentes de memoria, aunque tengan el mismo valor después de la asignación.

También es posible definir una referencia a una variable utilizando el operador *ampersand* ('&'):

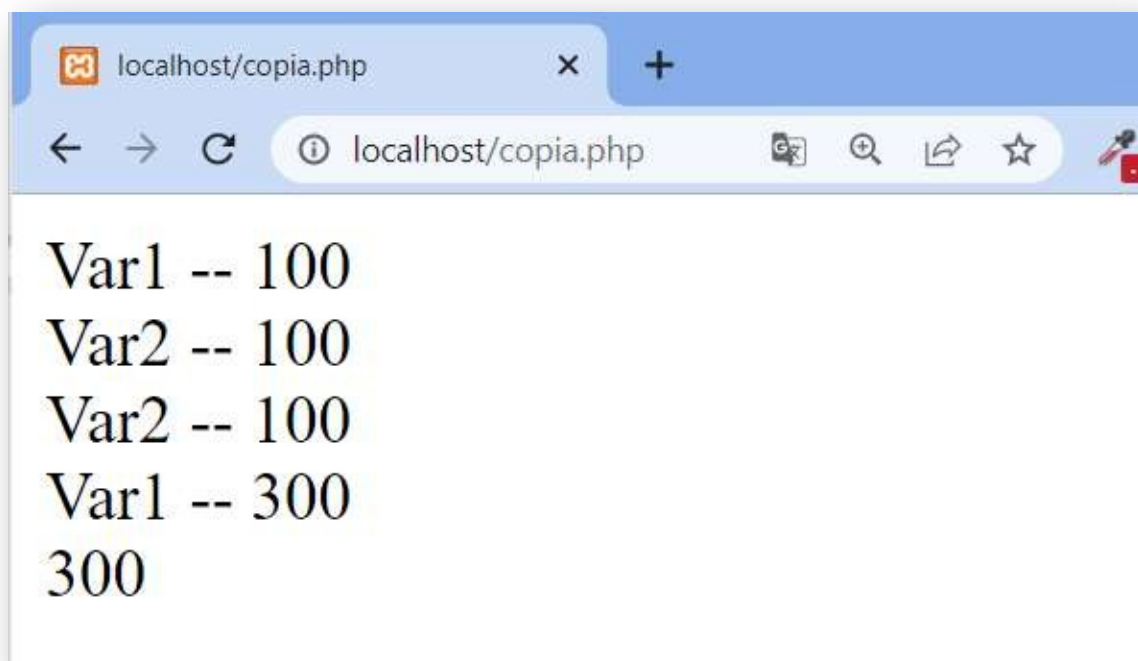
\$var2 = &\$var1

En este caso \$var2 no es una nueva variable con el valor de \$var1. Por el contrario, \$var2 apunta a la misma dirección de memoria que \$var1, de manera que \$var1 y \$var2 son en realidad dos nombres para el mismo dato.

Veamos el ejemplo siguiente:

```
1 <?php
2
3     $var1= 100;
4     $var2= &$var1; //Asignación por referencia
5
6     echo "Var1 -- $var1 <br>";
7     echo "Var2 -- $var2 <br>";
8
9     //Asignación por copia
10    $var3 = $var1;
11    echo "Var2 -- $var2 <br>"; //Muestra 100
12
13    //Cambia el valor de $var2
14    $var2 = 300;
15    echo "Var1 -- $var1 <br>"; //$var1 también cambia
16
17    //Este cambio no afecta a $var1
18    $var3 = 400;
19    echo $var1;
20
```

La salida en el navegador:



2.3.3.- Variables no inicializadas



Manual:

<https://www.php.net/manual/es/language.variables.basics.php>

Si se intenta utilizar una variable antes de asignarle un valor, se genera un error de tipo *E_NOTICE*, pero **no** se interrumpe la ejecución del *script*.



Si una variable no inicializada aparece dentro de una expresión, ésta se calcula tomando el valor por defecto para ese tipo de dato.

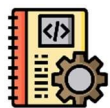
Ejemplo:

```
1  <?php
2      $var1 = 100;
3      $var3 = 100 + $var2; //$var2 no existe, se toma como 0
4      echo "$var3 <br>"; //mostraría 100
5
6      $var3 = 100 * $var2; //$var2 no existe, se toma como 0
7      echo "$var3 <br>"; //mostraría 0
```

El navegador mostraría lo siguiente:



2.3.4.- Constantes



Manual:

<https://www.php.net/manual/es/language.constants.php>

Para definir constantes se utiliza la función ***define()***, que recibe el nombre de la constante y el valor que queremos darle.

```
define("LIMITE", 1000);
```



Es habitual utilizar identificadores en mayúscula para las constantes.

2.3.5.- Tipos de datos escalares



Manual:

<https://www.php.net/manual/es/language.types.intro.php>

PHP ofrece cuatro tipos de datos escalares:

- *integer*
- *float*
- *boolean*
- *string*

A) NÚMEROS

Para representar números enteros se usa el tipo de dato ***integer***. El tamaño y los valores máximo y mínimo de un entero dependen de la plataforma, y se pueden conocer mediante las constantes:

- PHP_INT_SIZE
- PHP_INT_MAX
- PHP_INT_MIN

Para números reales, se utiliza el tipo ***float***. Suele ofrecer una precisión de 14 decimales (aunque dependerá de la plataforma).



La conversión entre ***integer*** y ***float*** es automática.

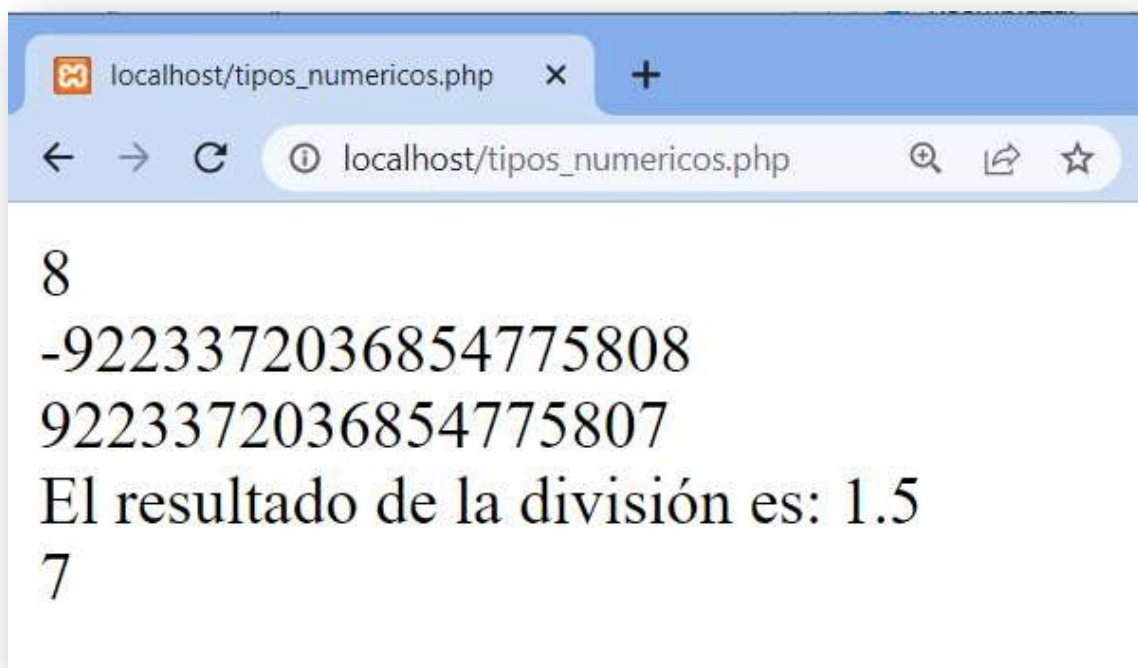


También se pueden utilizar los operadores de conversión **(int)** o **(float)**

Veamos este ejemplo:

```
1 <?php
2     echo PHP_INT_SIZE.'<br>';
3     echo PHP_INT_MIN.'<br>';
4     echo PHP_INT_MAX.'<br>';
5
6     $a = 3/2; // la división entre enteros no da problemas
7     echo 'El resultado de la división es: '.$a.'<br>';
8
9     $b = 7.5;
10
11     $a = (int)$b; // casting a integer
12     echo $a.'<br>'; // 7, se trunca
```

La salida en el navegador es:

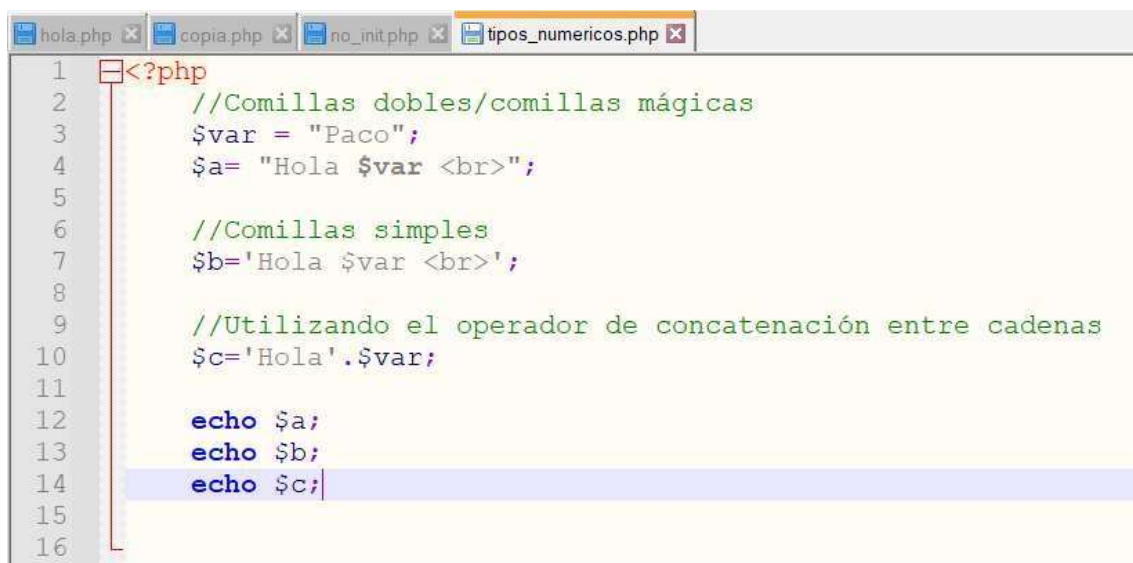


B) CADENAS

El tipo de dato *string* permite almacenar cadenas de caracteres. Para delimitar una cadena es posible utilizar comillas simples o dobles, pero hay una diferencia:

- Si se utilizan **comillas dobles** (también llamadas *comillas mágicas*, las variables que aparezcan dentro de la cadena se sustituirán por su valor.

Ejemplo:



```
1 <?php
2 //Comillas dobles/comillas mágicas
3 $var = "Paco";
4 $a= "Hola $var <br>";
5
6 //Comillas simples
7 $b='Hola $var <br>';
8
9 //Utilizando el operador de concatenación entre cadenas
10 $c='Hola'.$var;
11
12 echo $a;
13 echo $b;
14 echo $c;
```

La salida será:



C) BOOLEANOS

Este tipo de datos es para variables *booleanas*. Solo pueden tomar valores **TRUE** o **FALSE**.



Este es el tipo de dato que se obtiene, entre otros casos, como resultado de los operadores de comparación y se utiliza en sentencias condicionales y bucles.

Cuando se espera un valor *booleano* y se recibe otro tipo de dato, se aplican las reglas de conversión recogidas en el cuadro siguiente:

Tipo	Valor como <i>boolean</i>
integer	Si es 0 se toma FALSE, en otro caso como TRUE
float	Si es 0.0 se toma FALSE, en otro caso como TRUE
string	Si es una cadena vacía o "0", se toma como FALSE, en otro caso como TRUE.
Variables no inicializadas	FALSE
null	FALSE
array	Si no tiene elementos se toma FALSE, en otro caso como TRUE

D) OTROS TIPOS DE DATO (Tipos compuestos y especiales)

Además de los tipos de datos escalares en PHP también existen los siguientes tipos de datos:

1. *array*. Para representar colecciones de elementos.
2. *object*. PHP tiene soporte completo para la programación orientada a objetos
3. *callable*. Un tipo de dato especial para representar funciones de *callback*, funciones que se pasan a otras funciones.
4. *null*. El tipo de dato *null* representa una variable que no ha sido asignada. Solo puede tomar un único valor, NULL. Se considera que

una variable es de tipo *null* si se le asigna el valor NULL o no está inicializada.

5. *resource*. Este tipo de dato representa recursos externos, como una conexión a una base de datos.

A continuación, se presenta un cuadro resumen con los tipos de datos estudiados hasta ahora:

Tipo	Descripción
integer	Números enteros
float	Números reales en coma flotante
string	Cadenas de caracteres
boolean	Booleanos, TRUE o FALSE
array	Colección de elementos identificados
object	Un objeto es una instancia de una clase
callable	Para las funciones de <i>callback</i>
null	Para representar variables no asignadas
resource	Representa recursos externos

2.3.6.- Ámbito de las variables



Manual:

<https://www.php.net/manual/es/language.variables.scope.php>

El ámbito de una variable es la parte de código en que esta es visible.

- Una variable declarada en un fichero PHP está disponible en ese fichero y en los ficheros que se incluyan desde este.

- Las **funciones** definen un ámbito local, de manera que las variables que se declaran en las mismas solo son accesibles desde la propia función. Además, desde la función no se puede acceder a otras variables que no sean las locales o sus argumentos.
- Para definir **variables globales** hay dos opciones:
 - La palabra reservada **global**
 - La variable predefinida **\$_GLOBALS**

2.3.7.- Variables predefinidas



Manual:

<https://www.php.net/manual/es/language.variables.superglobals.php>

En PHP hay muchas variables predefinidas disponibles. Contienen información sobre el servidor, datos enviados por el cliente o variables de entorno.

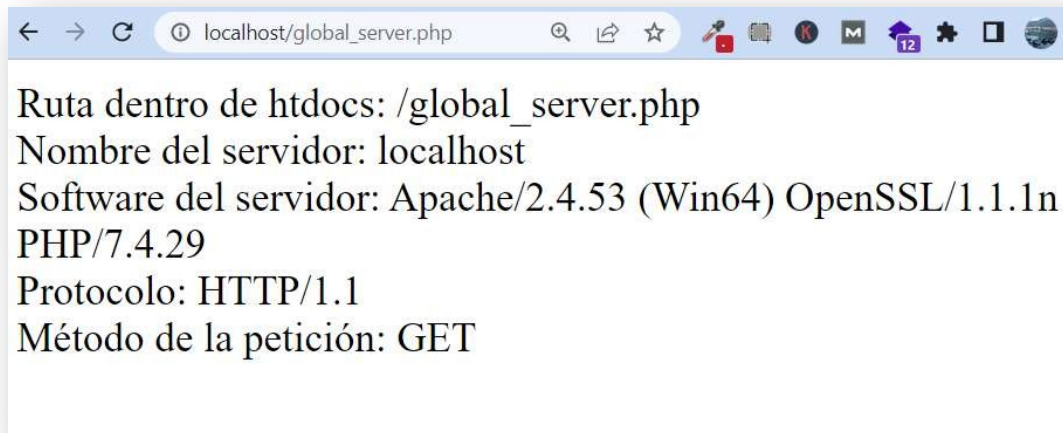
Dentro de las variables predefinidas hay un grupo de ellas, las *superglobales*, que **están disponibles en cualquier ámbito**. Cada una de ellas guarda información de un tipo.

Por ejemplo, en **\$_SERVER** hay información sobre el servidor en el que está alojada la página.

El script siguiente muestra algunos de los datos disponibles:

```
hola.php x copia.php x no_init.php x tipos_numericos.php x global_server.php x
1  <?php
2
3      echo "Ruta dentro de htdocs: " . $_SERVER['PHP_SELF'] . "<br>";
4      echo "Nombre del servidor: " . $_SERVER['SERVER_NAME'] . "<br>";
5      echo "Software del servidor: " . $_SERVER['SERVER_SOFTWARE'] . "<br>";
6      echo "Protocolo: " . $_SERVER['SERVER_PROTOCOL'] . "<br>";
7      echo "Método de la petición: " . $_SERVER['REQUEST_METHOD'] . "<br>";
```

La salida del navegador:



 Las variables *superglobales* son muy relevantes para el desarrollo de aplicaciones web y se irán poniendo en práctica a lo largo del curso.

A continuación, se muestran las variables:

Nombre	Descripción
\$GLOBALS	Variables globales definidas en la aplicación
\$_SERVER	Información sobre el servidor
\$_GET	Parámetros enviados con el método GET (en la URL)
\$_POST	Parámetros enviados con el método POST (formularios)
\$_FILES	Ficheros subidos al servidor
\$_COOKIE	<i>Cookies</i> enviadas por el cliente
\$_SESSION	Información de sesión
\$_REQUEST	Contiene la información de \$_GET, \$_POST y \$_COOKIE
\$_ENV	Variables de entorno

2.4.- Comentarios



Manual:

<https://www.php.net/manual/es/language.basic-syntax.comments.php>

En PHP se pueden utilizar comentarios:

- **De bloque**, encerrados entre “/*” y “*/”
- **De línea**, comenzando por “//” o por “#”.

Ejemplo:

```
1  <?php
2      //Comentario de línea
3      $a = 0; //comentario de línea
4
5      /*Comentario de
6      bloque*/
7
8      #Comentario de línea
```

¡ATENCIÓN!



Como cualquier lenguaje, comentar adecuadamente el código se considera una buena práctica de programación

2.5.- Expresiones y operadores en PHP

2.5.1.- Expresiones



Manual:

<https://www.php.net/manual/es/language.expressions.php>

Las expresiones son los bloques de construcción más importantes de PHP. En PHP casi todo lo que se escribe es una expresión. La manera más simple y acertada de definir lo que es una expresión es «cualquier cosa que tiene un valor». Utilizamos el signo igual para asignar un valor.

Las **formas más básicas de expresiones son las constantes y las variables**.

Ejemplo de expresión es cuando se escribe "**\$a = 5**", se está asignando '5' a \$a. '5', obviamente, tiene el valor 5, o en otras palabras, '5' es una expresión con el valor de 5 (en este caso, '5' es una constante entera).

Después de esta asignación, se espera que el valor de \$a sea 5 también, por lo que si se escribe \$b = \$a, se espera que esto se comporte tal como si se escribiera \$b = 5. En otras palabras, \$a es también una expresión con el valor 5. Si todo funciona bien, esto es exactamente lo que sucederá.

Un ejemplo de expresiones algo más complejo son las funciones. Pero las funciones se verán más adelante.

2.5.2.- Operadores



Manual:

<https://www.php.net/manual/es/language.operators.php>

PHP cuenta con los operadores habituales para operaciones aritméticas, lógicas, de manipulación de cadenas y demás.

Entre los operadores de comparación cabe señalar los operadores "==" y "!=", llamados Idéntico y No idéntico, que no están presentes en todos los lenguajes.



El operador Idéntico se usa para comparar dos expresiones y se evalúa como verdadero cuando las dos expresiones tienen el mismo valor y además el mismo tipo de dato.

Se diferencia del operador Igual, "=", en que este, cuando las expresiones no tienen el mismo tipo de dato, intenta convertirlas antes de compararlas.

Ejemplo:

```
1 <?php
2
3     $a = 3;
4     $b = "3";
5
6     //operador de comparación "Igual"
7     if($a == $b){
8         echo "Son iguales <br>";
9     }else{
10        echo "No son iguales <br>";
11    }
12    // Operador de comparación "Idéntico"
13    if ($a === $b){
14        echo "Con idénticos <br>";
15    }else{
16        echo "No son idénticos <br>";
17    }
```

La salida es:



En un lenguaje de programación los valores que vamos a relacionar mediante operadores pueden estar contenidos en variables, o escritos en el propio código. Los operadores toman esos valores y los modifican generando nuevos valores. A veces son conocidas operaciones matemáticas, pero a veces son operaciones lógicas o de asignación, por poner varios ejemplos.

- **Operador de asignación (=).** Lo utilizamos para asignar un valor a una variable.
- **Operadores aritméticos.** Nos permiten realizar operaciones numéricas con nuestras variables. Son los más fáciles de asimilar, porque generalmente todos conocemos esas operaciones.
 - + Suma, ejemplo: `$suma = $a + $b;`
 - - Resta, ejemplo: `$resta = $a - $b;`
 - * Multiplicación, ejemplo: `$mul = $a * $b;`
 - / División, ejemplo: `$divi= $a / $b;`
 - % Módulo (resto de la división) , ejemplo: `$resto=$a%$b;`
 - ** Exponenciación (2 ** 3, elevar 2 la a tercera potencia), ejemplo: `$expo =$a**3;`
- **Operador de asignación (=).** Lo utilizamos para asignar un valor a una variable. También, PHP incluye una serie de operadores de asignación con una operación asociada, que nos permiten realizar esa operación y una asignación en un único paso.
 - += Suma y asignación: `$suma+=5;`
 - -= Resta y asignación, ejemplo: `$resta-=10;`
 - *= Multiplicación y asignación, ejemplo: `$fact*=$a;`
 - /= División y asignación, ejemplo: `$divi/=10;`
 - %= Módulo y asignación, ejemplo: `$resul%=$b;`

- .= Concatenación y asignación, ejemplo: \$cadena.= "concateno";
- **Operadores de comparación.** Se utilizan principalmente en nuestras condiciones para comparar dos variables y verificar si cumple o no la propiedad del operador.
 - == Comprueba si son iguales
 - != Comprueba si son distintos
 - === Comprueba si son iguales y de exactamente el mismo tipo
 - !== Comprueba si son distintos o de distinto tipo
 - <> Diferente (igual que !=)
 - < Menor que, comprueba si un valor es menor que otro
 - > Mayor que
 - <= Menor o igual
 - >= Mayor o igual
 - <=> Comparador de orden. Por ejemplo: \$a<=> \$b devuelve un [integer](#) menor que, igual a, o mayor que cero cuando **\$a es respectivamente menor que, igual a, o mayor que \$b**. Disponible a partir de PHP 7.
 - ?? uno o el otro (PHP 7). Devuelve el primer operando de izquierda a derecha que exista y no sea NULL. NULL si no hay valores definidos y no son NULL.
Por ejemplo \$a ?? \$b ?? \$c.
- **Operadores lógicos.** Los operadores lógicos sirven para realizar operaciones lógicas, valga la redundancia. Son operaciones que al final van a devolver un "sí" o un "no", positivo o negativo. Se usan en combinación con los operadores de comparación cuando la expresión de la condición lo requiere.
 - **and** Operación lógica "y", será verdadero si ambos son verdaderos.
 - **or** Operación lógica "o", será verdadero si uno de ellos es verdadero
 - **xor** Operación lógica "xor", será verdadero si uno de ellos es verdadero, pero no ambos.
 - **!** Operación de negación, si era verdadero pasa a falso y viceversa.
 - **&&** Operación lógica "y"
 - **||** Operación lógica "o"

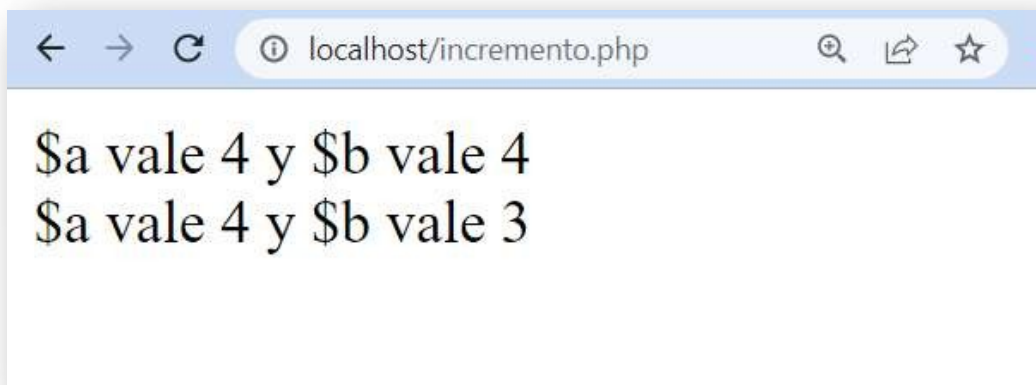
- **Operadores de incremento.** Sirven para aumentar o disminuir en una unidad el valor de una variable. Son atajos para hacer una operación típica en los recorridos de bucles, de aumentar o disminuir un valor que usamos para llevar la cuenta de las iteraciones realizadas.
 - **++\$a** Pre-incremento. Primero suma y luego asigna.
 - **\$a++** Post-incremento. Primero asigna y luego suma.
 - **--\$a** Pre-decremento. Primero resta y luego asigna.
 - **\$a--** Post-decremento. Primero asigna y luego resta.

Estos operadores son interesantes, porque realizan dos cosas, igual que los operadores de asignación combinada que vimos antes. Por un lado, un decremento o incremento y por otro lado se devuelven ellos mismos como resultado del operador. Lo curioso es que existen los "pre" y los "post" y es que funcionan de manera un poco distinta.

Cuando tenemos un pre-incremento o pre-decremento, primero se realiza el incremento/decremento y luego se devuelve ese valor. Con el post-incremento o post-decremento, primero se devuelve el valor original y luego se realiza el incremento/decremento. Para acabar de entenderlo es mejor examinar este código:

```
1  <?php
2
3      $a = 3;
4      $b = ++$a;
5
6      // $a vale 4 y $b vale 4
7      echo "\$a vale $a y \$b vale $b <br>";
8
9      $a = 3;
10     $b = $a++;
11
12     // $a vale 4 y $b vale 3
13     echo "\$a vale $a y \$b vale $b <br>";
14
```

La salida es:



- **Operadores de cadenas.** Cuando trabajamos con cadenas de caracteres tenemos un operador especial que es el de la concatenación. Sirve para unir una cadena a la otra.
 - . Concatenación

```
1  <?php
2
3      $saludo = "Hola ";
4      $nombre = "Mi querido amigo";
5
6      //Salida: "Hola Mi querido amigo"
7      $saludoCompleto = $saludo . $nombre;
8
9      echo $saludoCompleto;
```



- **Operadores para arrays.** Se utilizan para comparar arrays. En el ejemplo suponemos que $\$a$ y $\$b$ son arrays. Más adelante se verán los arrays.

Ejemplo	Nombre	Resultado
$\$a + \b	Unión	Unión de $\$a$ y $\$b$. El operador $+$ devuelve el array del lado derecho añadido al array del lado izquierdo; para las claves que existan en ambos arrays, serán utilizados los elementos del array de la izquierda y serán ignorados los elementos correspondientes del array de la derecha
$\$a == \b	Igualdad	TRUE si $\$a$ i $\$b$ tienen las mismas parejas clave/valor.
$\$a === \b	Identidad	TRUE si $\$a$ y $\$b$ tienen las mismas parejas clave/valor en el mismo orden y de los mismos tipos.
$\$a != \b	Desigualdad	TRUE si $\$a$ no es igual a $\$b$.
$\$a <> \b	Desigualdad	TRUE si $\$a$ no es igual a $\$b$.
$\$a !== \b	No-identidad	TRUE si $\$a$ no es idéntica a $\$b$.

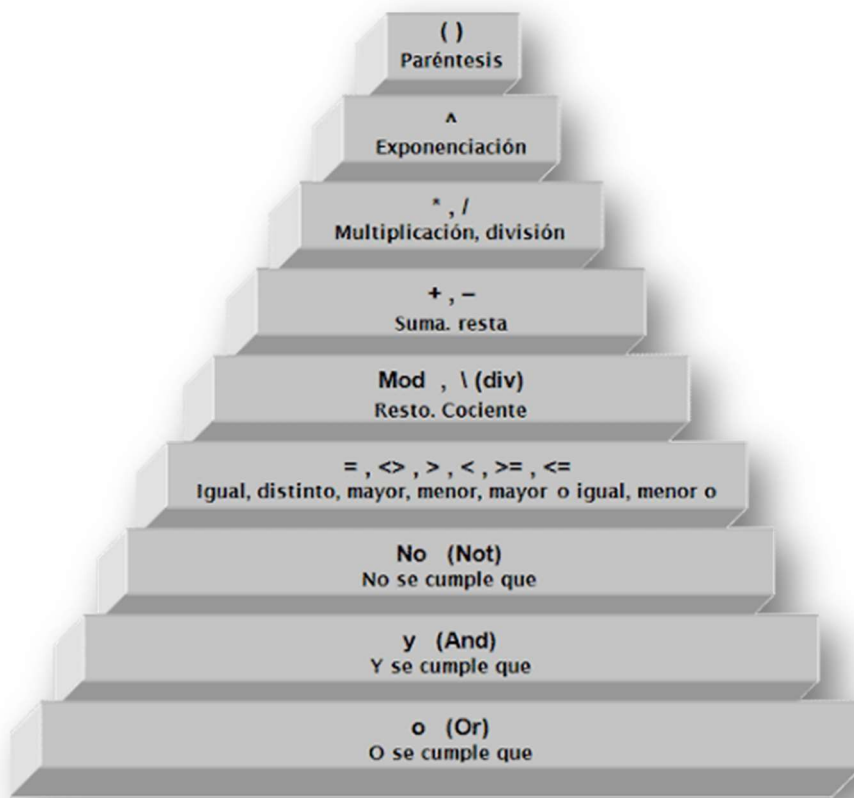
2.5.3.- Orden de prioridad



Manual:

<https://www.php.net/manual/es/language.operators.precedence.php>

Los operadores lógicos y matemáticos tienen un orden de prioridad o precedencia. Este es un esquema general que indica el orden en que deben evaluarse en la mayoría de los lenguajes de programación.



Lo aconsejable es que cuando se tenga una expresión con varios operadores y operandos se utilicen los paréntesis, **así no dará lugar a dudas, y además no tendrá distintas interpretaciones.**

2.6.- Funciones nativas de PHP

En PHP ya existen en memoria funciones que nos permiten realizar distintas operaciones al invocarlas. A continuación, veremos algunas de las funciones más utilizadas:

2.6.1.- La más utilizadas (Resumen)

Funciones de variables	
isset(\$var)	TRUE si la variable está inicializada y no es NULL
is_null(\$var)	TRUE si la variable es NULL
empty(\$var)	TRUE si la variable no está inicializada o su valor es FALSE
is_int(\$var), is_float(\$var), is_bool(\$var), is_array(\$var)	Para comprobar el tipo de dato de \$var
intval(\$var), floatval(\$var), boolvar(\$var), strval(\$var)	Para obtener el valor de \$var como otro tipo de dato

Funciones de cadenas	
strlen(\$cad)	Devuelve la longitud de \$cad
explode(\$cad, \$token)	Parte una cadena utilizando \$token como separador. Devuelve un array de cadenas
implode(\$token, \$array)	Crea una cadena larga a partir de un array de cadenas, entre cadena y cadena se introduce \$token
strcmp(\$cad1, \$cad2)	Compara las dos cadenas. Devuelve 0 si son iguales, -1 si \$cad1 es menor y 1 si \$cad1 es mayor
strtolower(\$cad), strtoupper(\$cad)	Devuelven \$cad en mayúsculas o minúsculas, respectivamente
str(\$cad1, \$cad2)	Busca la primera ocurrencia de \$cad2 en \$cad1. Si no aparece devuelve FALSE, si aparece devuelve \$cad1 desde donde comienza la ocurrencia

Funciones de arrays	
ksort(\$arr), krsort(\$arr)	Ordena el array por clave en orden ascendente o descendente
sort(\$arr), rsort(\$arr)	Ordena el array por valor en orden ascendente o descendente
array_values(\$arr)	Devuelve los valores de \$arr
array_keys(\$arr)	Devuelve las claves de \$arr
array_key_exists(\$arr, \$cla)	Devuelve verdadero si algún elemento de \$arr tiene clave \$cla
count(\$arr)	Devuelve el número de elementos del array



En ANEXOS se presentan muchas funciones predefinidas en PHP. Su uso puede facilitaros mucho la resolución de un problema mediante un script en PHP.

3.- Glosario de términos

Array. Es un tipo de dato compuesto habitual en los lenguajes de programación. Aunque los detalles varían entre lenguajes, en general, se parecen a vectores o listas ordenadas.

Bucle. Estructura de programación que permite repetir instrucciones

Estructura condicional. Posibilita ejecutar o no una instrucción según se cumpla una condición

Función. Conjunto de instrucciones que realiza una tarea concreta

Librería. Las funciones relacionadas entre sí se agrupan en librerías o bibliotecas.

Programación orientada a objetos. Paradigma de programación basado en la idea de objetos, elementos que agrupan variables y funciones.

Script. Programa sencillo, habitualmente ejecutado por un intérprete en lugar de ser compilado.

Variable. Posición en la memoria del ordenador identificada por un nombre. La variable almacena datos. Estos datos son el valor de la variable.

4.- ANEXOS – AMPLIACIÓN DE FUNCIONES NATIVAS PHP



Listado de funciones nativas – PHP Manual:

<https://www.php.net/manual/es/indexes.functions.php>

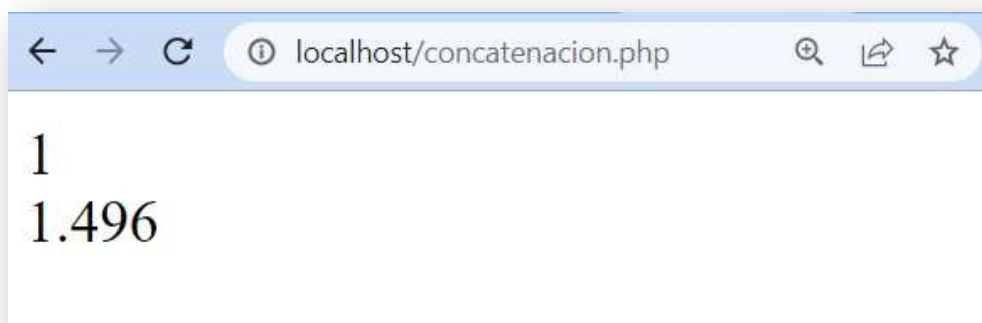
A continuación, se presentan funciones nativas de PHP que puede ser de ayuda para programar scripts en este lenguaje:

4.1.- Numéricas

- **round()**

Esta función nos permitirá hacer redondeo, osea recibirá como parámetro un número y si el valor después del punto (o de la coma) es menor a 50 redondeará para abajo, de lo contrario lo hará un número más. Se pueden añadir número de dígitos de precisión poniendo **round(\$num, \$precision)**

```
1  <?php
2
3      //Redondeo por defecto
4      $decimal = 1.4956;
5      $decimal_redondeado = round($decimal);
6
7      echo $decimal_redondeado."<br>";
8
9      //Redondeo a 3 decimales
10     $decimal_redondeado = round($decimal,3);
11     echo $decimal_redondeado."<br>";
12
13
```

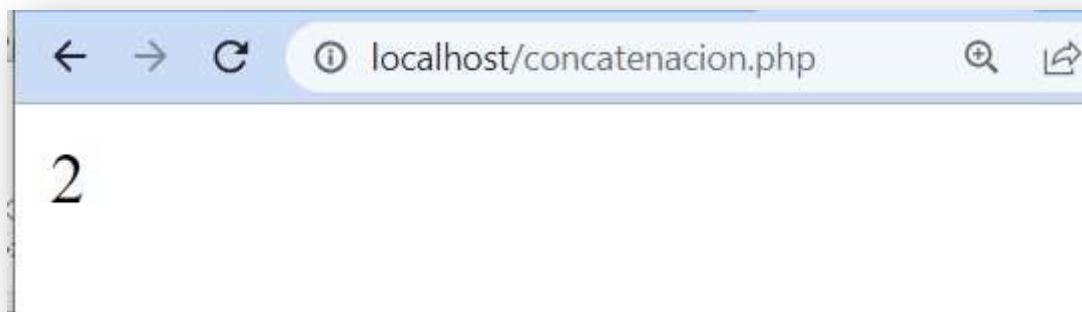


← → ↻ ⓘ localhost/concatenacion.php 🔍 📄 ☆

1
1.496

Pero si en cambio modificamos el número decimal por 1.75:

```
1 <?php
2
3     $decimal = 1.75;
4     $decimal_redondeado = round($decimal);
5     echo $decimal_redondeado;
6
7
8
```



Entonces el resultado será: 2



Esta función es muy útil a la hora de obtener promedios.

- **floor()**

Esta función también hará un redondeo, pero siempre para abajo, es decir que si tenemos esto:

```
<?php
    $decimal = 1.50;
    $decimal_redondeado = floor($decimal);
    echo $decimal_redondeado;
?>
```

Nos devolverá por pantalla: 1

- **ceil()**

Si en cambio queremos que el redondeo sea siempre para arriba deberíamos usar **ceil()**:

```
<?php
    $decimal = 1.50;
    $decimal_redondeado = ceil($decimal);
    echo $decimal_redondeado;
?>
```

En donde el resultado será por supuesto: 2

- **rand()**

La función **rand()** es una de las más útiles que hay. Esta función generará un número al azar que irá desde un rango desde/hasta. Por ejemplo si quisiéramos devolver un número desde el 1 al 9 (inclusive):

```
<?php
    $numero_al_azar = rand(1, 9);
    echo $numero_al_azar;
?>
```

A mí me devolvió: 3

Pero puede devolver cualquiera de los números de los rangos comprendidos.

4.2.- Cadenas

Hay que tener en cuenta que el primer carácter de una cadena ocupa la posición 0.

- **strtoupper()**

Convierte una cadena en mayúscula;

```
<?php

    $palabra = 'Me encantan las arepas venezolanas.';
    $palabra_mayuscula = strtoupper($palabra);
    echo $palabra_mayuscula;
?>
```

Esto devolverá por pantalla: ME ENCANTAN LAS AREPAS VENEZOLANAS.

- **mb_strtoupper(\$cadena, [, string \$encoding = mb_internal_encoding()])**

Convierte una cadena en mayúscula, a diferencia de lo anterior la cadena **puede ser una cadena de texto con tildes.**

El parámetro encoding es la codificación de caracteres. Si es omitido, será usado el valor de la codificación de caracteres interna.

A diferencia de **strtoupper()**, la conversión se lleva a cabo según los fundamentos de las propiedades de **los caracteres Unicode**. Esta función no se ve afectado por la configuración regional, por lo que puede convertir cualquier carácter que tenga la propiedad alfabética, como la a con acento (á).

Prueba lo siguiente:

```
$palabra= "Martín Martínez León";  
echo "<br>La cadena en mayúscula es: ". mb_strtoupper($palabra);  
echo "<br>La cadena en mayúscula es: ". strtoupper($palabra);
```

En la salida la segunda línea no convierte los acentos a mayúsculas

La cadena en mayúscula es: MARTÍN MARTÍNEZ LEÓN

La cadena en mayúscula es: MARTÍN MARTÍNEZ LEÓN

- **strtolower()**

Esta función es lo contrario a la anterior, convertirá una cadena pero en minúsculas:

```
<?php  
$palabra = 'No GRITES, habla bajito.';  
$palabra_minuscula = strtolower($palabra);  
echo $palabra_minuscula;  
?>
```

Esto nos mostrará por pantalla: no grites, habla bajito.

- **mb_strtolower(\$cadena, [, string \$encoding = mb_internal_encoding()])**

Convierte una cadena en minúscula, a diferencia de lo anterior la cadena **puede ser una cadena de texto con tildes.** El parámetro encoding es la codificación de caracteres. Si es omitido, será usado el valor de la

codificación de caracteres interna. Nos va a permitir convertir cualquier carácter.

Prueba lo siguiente:

```
$palabra= "MARTÍN MARTÍNEZ LEÓN";  
echo "<br>La cadena en minúscula es: ". mb_strtolower($palabra);  
echo "<br>La cadena en minúscula es: ". strtolower($palabra);
```

En la salida la segunda línea no convierte los acentos a minúsculas

La cadena en minúscula es: martín martínez león

La cadena en minúscula es: mart??n mart??nez le??n

- **ucfirst()**

En este caso, esta función sólo nos convertirá la primera letra de una cadena en mayúscula:

```
<?php  
$palabra = 'fernando';  
$nombre = ucfirst($palabra);  
echo $nombre;  
?>
```

Devolverá por pantalla: Fernando

Si la primera letra es acentuada, necesitamos extraer el primer carácter y convertirlo a mayúscula (`mb_strtoupper(mb_substr($nombr, 0, 1))`), y luego concatenar con el resto de la cadena (`mb_substr($nombr, 1)`):

```
$nombr = "área del círculo";  
echo "<br>nombr = " . ucwords($nombr);  
echo "<br>nombr = " . ucfirst($nombr);  
echo "<br>nombr = " . mb_convert_case($nombr, MB_CASE_TITLE);  
echo "<br>nombr = " . mb_strtoupper(mb_substr($nombr, 0, 1)) .  
mb_substr($nombr, 1);
```

nombr = área Del Círculo

nombr = área del círculo

nombr = Área Del Círculo

nombr = Área del círculo

- **ucwords()**

En este caso, esta función recibirá una cadena y convertirá en mayúscula los primeros caracteres de cada palabra:

```
<?php
    $palabra = 'fernando gonzalo gaitan';
    $nombre_completo = ucwords($palabra);
    echo $nombre_completo;
?>
```

Esto va a imprimir: Fernando Gonzalo Gaitan

- **mb_convert_case**(string \$str , int \$mode [, string \$encoding = mb_internal_encoding()])

Realiza una conversión a mayúsculas/minúsculas en un string, de acuerdo al modo especificado por mode. La conversión se lleva a cabo según los fundamentos de las propiedades de **los caracteres Unicode**. De esta forma, esta función no se ve afectada por las configuraciones regionales, ya que puede convertir cualquier carácter que contenga la propiedad 'alfabética', como la A con diéresis (Ä).

El modo de conversión. Puede ser MB_CASE_UPPER, MB_CASE_LOWER, o MB_CASE_TITLE.

```
$palabra= "Mi nombre es Martín Martín León";
echo "<br>Cambio mayus por minusculas: ";
mb_convert_case($palabra, MB_CASE_LOWER);
echo "<br>Cambio minus por mayus: ". mb_convert_case($palabra,
MB_CASE_UPPER);
echo "<br>Cambio a tipo título: ". mb_convert_case($palabra,
MB_CASE_TITLE);
```

Devolverá por pantalla:

Cambio mayus por minusculas: mi nombre es martín martín león
Cambio minus por mayus: MI NOMBRE ES MARTÍN MARTÍN LEÓN
Cambio a tipo título: Mi Nombre Es Martín Martín León

- **strlen()**

Nos dará la cantidad de caracteres que tiene una cadena:

```
<?php
    $cancion = 'De aquel amor de música ligera, nada nos libra, nada
    más queda';
    $cantidad_de_caracteres = strlen($cancion);
    echo $cantidad_de_caracteres;
?>
```

Esto da como resultado: 64

- **mb_strlen (string \$str [, string \$encoding = mb_internal_encoding()])**

Nos dará la cantidad de caracteres que tiene una cadena que contiene **caracteres Unicode**.

```
$palabra= "Martín Martín León";
echo "<br>Longitud: ". strlen($palabra);
echo "<br>Longitud unicode: ". mb_strlen($palabra);
```

Devolverá por pantalla:

Longitud: 21

Longitud unicode: 18

- **substr()**

Esta función nos permitirá seleccionar cuántos caracteres queremos mostrar de una cadena. Recibe tres parámetros, el primero es la cadena en sí, el segundo es a partir de qué carácter queremos que se muestre (empieza desde 0) y el tercero es el número de caracteres:

```
<?php
    $cancion = 'De aquel amor de música ligera, nada nos libra, nada
    más queda';
    $cancion_cortada = substr($cancion, 0, 11);
    echo $cancion_cortada;
?>
```

Esto nos dará como resultado: De aquel am

Si el inicio es negativo, la cadena devuelta empezará en el inicio que pongamos pero contando desde el final de la cadena.

- **mb_substr (string \$str , int \$start [, int \$length = NULL [, string \$encoding = mb_internal_encoding()]])**

Lo mismo, pero con caracteres unicode

```
$palabra= "Martín aé Martín úí León";  
echo "<br>De la posición 5, extraigo 10: ". substr($palabra, 5, 10);  
echo "<br>De la posición 5, extraigo 10 con mb_string: ".  
mb_substr($palabra, 5, 10);
```

Esto nos dará como resultado

De la posición 5, extraigo 10: n aé Ma

De la posición 5, extraigo 10 con mb_string: n aé Martí

- **mb_internal_encoding ([string \$encoding = mb_internal_encoding()])**

Establece/obtiene la codificación de caracteres interna. Por defecto es UTF-8

```
echo "<br>mb_internal_encoding: ".mb_internal_encoding();
```

<https://www.php.net/manual/es/function.mb-internal-encoding.php>

- **trim()**

Ésta es otra de las funciones más usadas de PHP, se trata de una función simple, pero muy útil, la misma **cortará los espacios en las cadenas tanto adelante como atrás:**

```
<?php  
$cadena = ' Hola ';  
$cadena_cortada = trim($cadena);  
echo $cadena_cortada;  
?>
```

Para comprobar su correcto funcionamiento podemos utilizar la función **strlen()** y ver el comportamiento de **trim()**:

```
<?php
    $cadena = ' Hola ';
    $cadena_cortada = trim($cadena);
    echo strlen($cadena) . '<br />';
    echo strlen($cadena_cortada) . '<br />';
?>
```

Esto por pantalla devolverá:

6

4

Como vemos la segunda variable tendrá sólo 4 caracteres ya que los caracteres de adelante y atrás se eliminarán.

Sin embargo **trim()** no sólo funciona para eliminar espacios al principio y al final, puede eliminar cualquier tipo de carácter si le agregamos **un segundo parámetro** con el mismo:

```
<?php
    $cadena = '///Hola///';
    $cadena_cortada = trim($cadena, '/');
    echo $cadena_cortada;
?>
```

Al imprimir el resultado por pantalla veremos: Hola

Los otros caracteres predefinidos pueden ser estos:

- "\0" - NULL
- "\t" - tab
- "\n" - new line
- "\x0B" - vertical tab
- "\r" - carriage return
- " " - ordinary white space

<https://www.php.net/manual/es/function.trim.php>

- **ltrim (\$cadena)** - Elimina espacios en blanco u otros caracteres predefinidos del lado izquierdo de una cadena
- **rtrim (\$cadena)** - Elimina espacios en blanco u otros caracteres predefinidos del lado derecho de una cadena

- **substr_count(\$cadena, \$cadenaabuscar);** Cuenta el número de apariciones de la cadena que escribimos en cadena a buscar. Muy útil si se desea preguntar si una cadena está contenida en otra cadena.

```
<?php
    $text = 'This is a test';
    echo strlen($text); // 14
    echo substr_count($text, 'is'); // cuenta 2
?>
```

- **str_replace()**

La función **str_replace()** nos permitirá buscar una cadena específica y reemplazarla por otra,

str_replace(buscar, reemplazar, lacadena, count)

- buscar. Especifica el valor a encontrar
- reemplazar. Especifica el valor para reemplazar el valor en buscar
- cadena. Especifica la cadena que se buscará
- contar. Una variable que cuenta el número de reemplazos.
(opcional)

por ejemplo podríamos reemplazar guiones bajo por guiones del medio:

```
<?php
    $frase = 'Marge_creo_que_odio_a_Mickael_Jackson.';
    $frase_modificada = str_replace('_', '-', $frase, $c);
    echo "<br>Frase ". $frase_modificada;
    echo "<br>Cambios ". $c;?
>
```

El resultado será: Marge-creo-que-odio-a-Mickael-Jackson. Cambios 6

Pero como dijimos antes, se pueden modificar cadenas, o sea no sólo un carácter:

```
<?php
    $frase = 'Una manzana y una manzana son dos manzanas';
    $frase_modificada = str_replace('manzana', 'naranja', $frase);
    echo $frase_modificada;
?>
```

Esto imprime por pantalla: Una naranja y una naranja son dos naranjas

- **strtr (\$cadena , \$from , \$to)**

Convierte caracteres o reemplaza substrings.

Si se dan tres argumentos, esta función devuelve una copia de cadena, donde todas las apariciones de cada carácter (byte simple) en from han sido convertidas al carácter correspondiente en to, es decir, **todas las apariciones de \$from[\$n] han sido reemplazadas con \$to[\$n]**, donde \$n es un índice válido en ambos argumentos.

Si from y to tienen distinta longitud, se ignoran los caracteres extra del string más largo. La longitud de str será la misma que la del valor devuelto.

Prueba lo siguiente:

```
$cadena= "Martín aé Martín úí León";  
echo "<br>Cambiar la a por A, la e por E y la t por T: ".strstr(  
$cadena,'aet','AET');
```

Visualizará:

Cambiar la a por A, y la e por E y la t por T: MArTín aé MArTín úí León

<https://www.php.net/manual/es/function.strtr.php>

- **La función strstr** — Encuentra la primera aparición de un string

<https://www.php.net/manual/es/function strstr.php>

strstr (string \$cadena1 , [string](#) \$cadena2) : string

Devuelve parte del string cadena1 iniciando desde, e incluyendo, la primera aparición de cadena2 hasta el final de cadena1.

Nota: Esta función es sensible a mayúsculas. **Para búsquedas sin importar las mayúsculas, usar [strstr\(\)](#).**

```
$cadena1 = "Probando a buscar una cadena en otra cadena";  
$cadena2 = "en otra";  
$resultado = strstr($cadena1, $cadena2);  
echo "<br>Resultado: " . $resultado;  
if (strstr($cadena1, $cadena2)) {  
    echo "<br>" . $cadena2 . " Contenida en: " . $cadena1;  
}
```

Devuelve:

Resultado: en otra cadena
en otra Contenedora en: Probando a buscar una cadena en otra
cadena

- Las funciones [stripes](#), [strpos](#) y [strstr](#) sirven para **encontrar subcadenas**.

stripes — Encuentra la posición de la primera ocurrencia de un substring en un string

stripes — Encuentra la posición de la primera aparición de un substring en un string sin considerar mayúsculas ni minúsculas

A diferencia de [stripes\(\)](#), **stripes()** no considera las mayúsculas ni las minúsculas.

<https://www.php.net/manual/es/function.strpos.php>

```
<?php
    $mystring = 'abc';
    $findme = 'a';
    $pos = strpos($mystring, $findme);

    // Nótese el uso de ===. Puesto que == simple no funcionará como
    // se espera
    // porque la posición de 'a' está en el 1º (primer) carácter.
    if ($pos === false) {
        echo "La cadena '$findme' no fue encontrada en la cadena
        '$mystring'";
    } else {
        echo "La cadena '$findme' fue encontrada en la cadena
        '$mystring'";
        echo " y existe en la posición $pos";
    }
?>
```

- `str_contains` — Determine if a string contains a given substring (PHP 8)

```
<?php
$string = 'The lazy fox jumped over the fence';

if (str_contains($string, 'lazy')) {
    echo "The string 'lazy' was found in the string\n";
}

if (str_contains($string, 'Lazy')) {
    echo 'The string "Lazy" was found in the string';
} else {
    echo "'Lazy' was not found because the case does not match';
}

?>
```

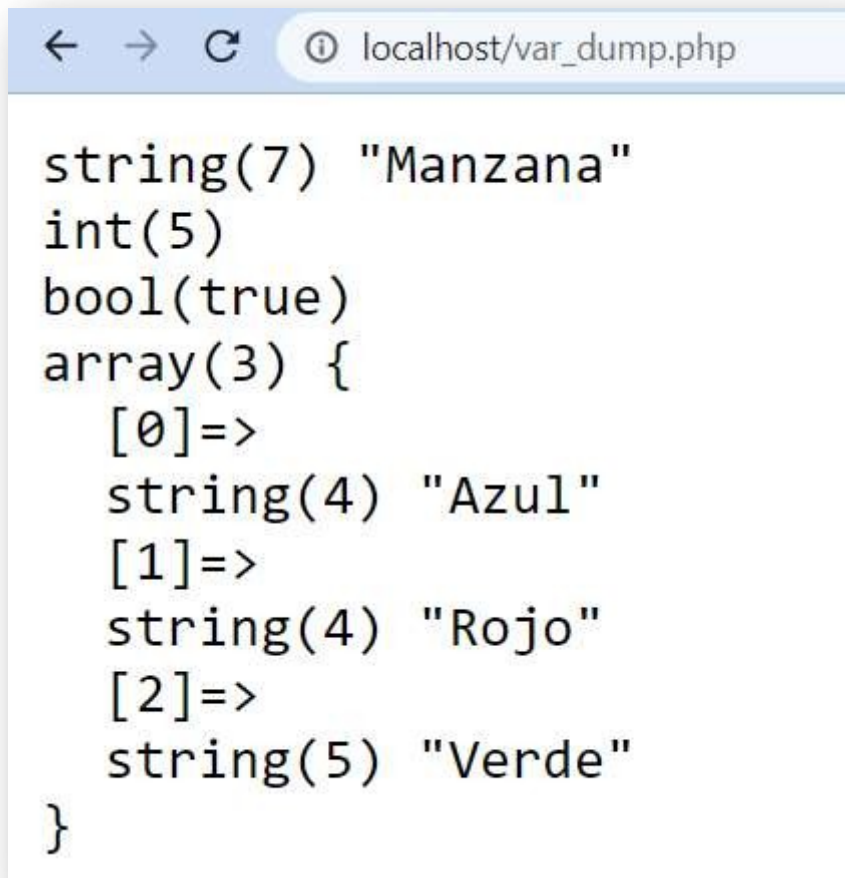
- `var_dump()`

La función **`var_dump()`** es muy útil en tiempo de desarrollo a la hora de testear datos ya sean de una variable o de lo que devuelve una función. Esta función nos devolverá la estructura de una variable cualquiera:

```
<?php
$cadena = 'Manzana';
$numero = 5;
$valor_logico = true;
$coleccion = array('Azul', 'Rojo', 'Verde');
echo '<pre>';
var_dump($cadena);
var_dump($numero);
var_dump($valor_logico);
var_dump($coleccion);
echo '</pre>';

?>
```

Esto devuelve:



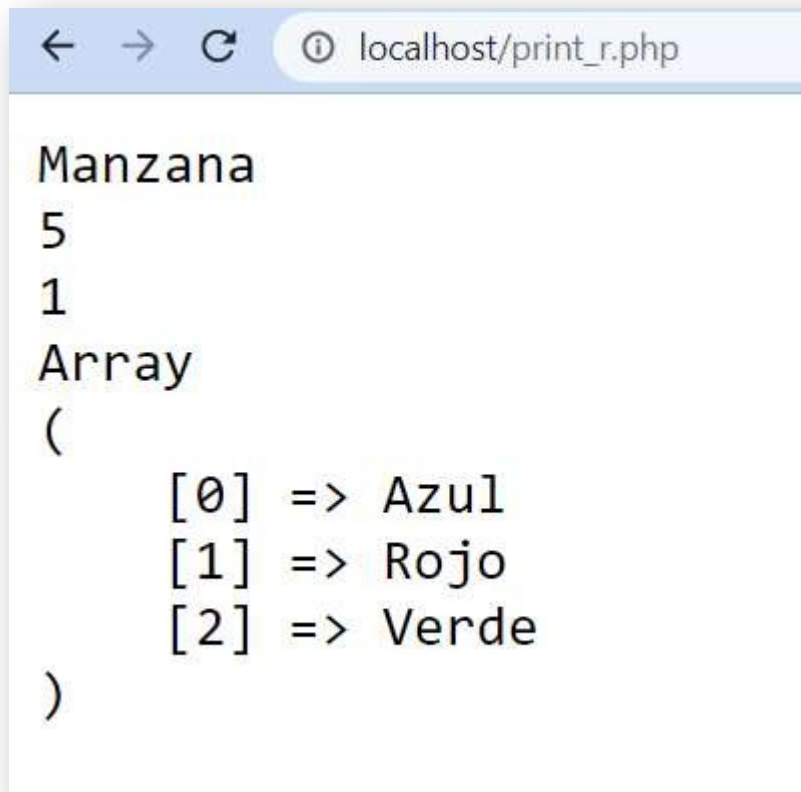
```
string(7) "Manzana"  
int(5)  
bool(true)  
array(3) {  
    [0]=>  
        string(4) "Azul"  
    [1]=>  
        string(4) "Rojo"  
    [2]=>  
        string(5) "Verde"  
}
```

- **print_r ()**

Muestra información sobre una variable de una manera que sea legible por humanos. Similar a var_dump.

```
$cadena= "Manzana";  
$numero = 5;  
$valor_logico = true;  
$coleccion = array('Azul', 'Rojo', 'Verde');  
echo '<pre>';  
    print_r($cadena);  
    echo '<br>';  
    print_r ($numero);  
    echo '<br>';  
    print_r($valor_logico);  
    echo '<br>';  
    print_r($coleccion);
```

```
echo '</pre>';
```



4.3.- De tipos

El false o null, no muestra nada a la hora de visualizar. Otro valor en una variable visualiza 1, que es true.

Funciones para convertir variables a otros tipos:

- [boolval\(\)](#) - Obtener el valor booleano de una variable

<https://www.php.net/manual/es/function.boolval.php>

```
$v=boolval(121212);  
echo "<br> v es ".$v ;  
$v=boolval(0);  
echo "<br> v es ".$v ;
```

- [intval\(\)](#) - Obtiene el valor entero de una variable

<https://www.php.net/manual/es/function.intval.php>

```
$v=intval(121212);  
echo "<br> v es ".$v ;  
$v=intval("ANBB");  
echo "<br> v es ".$v ;  
$v=intval("323244.656");  
echo "<br> v es ".$v ;
```

- [floatval\(\)](#) - Obtener el valor flotante de una variable

<https://www.php.net/manual/es/function.intval.php>

```
$v=floatval(121212);  
echo "<br> v es ".$v ;  
$v=floatval("ANBB");  
echo "<br> v es ".$v ;  
$v=floatval("323244.656");  
echo "<br> v es ".$v ;  
$v=floatval(123e-67);  
echo "<br> v es ".$v ;  
$v=floatval(14242.33e-6);  
echo "<br> v es ".$v ;
```

- [strval\(\)](#) - Obtiene el valor de cadena de una variable

<https://www.php.net/manual/es/function.strval.php>

```
echo "<br> -----";  
$v=strval(121212);  
echo "<br> v es ".$v ;  
$v=strval("ANBB afsfgsd");  
echo "<br> v es ".$v ;  
$v=strval("323244.656");  
echo "<br> v es ".$v ;  
$v=strval(123e-67);  
echo "<br> v es ".$v ;
```

- [settype\(\)](#) - Establece el tipo de una variable

<https://www.php.net/manual/es/function.settype.php>

```
$a = "sffsfsfr";  
$b = true;
```

```
$c="6767";  
settype($a, "integer");  
echo "<br> a es ".$a;  
settype($c, "integer");  
echo "<br> a es ".$a;  
echo "<br> b es ".$b;  
echo "<br> c es ".$c;  
settype($b, "string");  
echo "<br> b es ".$b;
```

Funciones para comprobar el tipo de las variables:

- [is_int\(\)](#) - Comprueba si el tipo de una variable es **integer**

<https://www.php.net/manual/es/function.is-int.php>

```
$v=121212;  
if(is_int($v)){  
    echo "<br> $v es int ";  
}else  
{  
    echo "<br> $v no es int ";  
}
```

```
$v="121212";  
if(is_int($v)){  
    echo "<br> $v es int ";  
}else  
{  
    echo "<br> $v no es int ";  
}
```

```
$v=121212.546;  
if(is_int($v)){  
    echo "<br> $v es int ";  
}else  
{  
    echo "<br> $v no es int ";  
}
```

Muestra:

121212 es int
121212 no es int
121212.546 no es int

- [is_bool\(\)](https://www.php.net/manual/es/function.is-bool.php) - Comprueba si una variable es de tipo booleano

<https://www.php.net/manual/es/function.is-bool.php>

```
$v=111;
if(is_bool($v)){
    echo "<br> $v es true ";
}else
{
    echo "<br> $v es false ";
}

$v=false;
if(is_bool($v)){
    echo "<br> $v es true ";
}else
{
    echo "<br> $v es false ";
}
```

- [is_null\(\)](https://www.php.net/manual/es/function.is-null.php) - Comprueba si una variable es NULL

<https://www.php.net/manual/es/function.is-null.php>

```
echo "<br>-----";
$v = 111;
if (is_null($v)) {
    echo "<br> is null $v";
} else {
    echo "<br> no null $v";
}

$v = false;
if (is_null($v)) {
    echo "<br> is null $v";
} else {
```

```
    echo "<br> no null $v";  
}
```

```
$v = 0;  
if (is_null($v)) {  
    echo "<br> is null $v";  
} else {  
    echo "<br> no null $v";  
}
```

```
$v = null;  
if (is_null($v)) {  
    echo "<br> is null $v";  
} else {  
    echo "<br> no null $v";  
}
```

- [is_float\(\)](https://www.php.net/manual/es/function.is-float.php) - Comprueba si el tipo de una variable es float

<https://www.php.net/manual/es/function.is-float.php>

```
echo "<br>-----";  
$v = 123.11;  
if (is_float($v)) {  
    echo "<br> is float $v";  
} else {  
    echo "<br> no float $v";  
}
```

```
$v = "123.11";  
if (is_float($v)) {  
    echo "<br> is float $v";  
} else {  
    echo "<br> no float $v";  
}
```

- [is_numeric\(\)](https://www.php.net/manual/es/function.is-numeric.php) - Comprueba si una variable **es un número o un string numérico**

<https://www.php.net/manual/es/function.is-numeric.php>

```
echo "<br>-----";
```

```
$v = "121212";  
if (is_numeric($v)) {  
    echo "<br> $v es numeric ";  
} else {  
    echo "<br> $v no es numeric ";  
}
```

```
$v = 121212;  
if (is_numeric($v)) {  
    echo "<br> $v es numeric ";  
} else {  
    echo "<br> $v no es numeric ";  
}
```

```
$v = "dadsadf";  
if (is_numeric($v)) {  
    echo "<br> $v es numeric ";  
} else {  
    echo "<br> $v no es numeric ";  
}
```

- [is_string\(\)](https://www.php.net/manual/es/function.is-string.php) - Comprueba si una variable es de tipo string

<https://www.php.net/manual/es/function.is-string.php>

```
echo "<br>-----";  
$v = "121212";  
if (is_string($v)) {  
    echo "<br> $v es string";  
} else {  
    echo "<br> $v no es string";  
}
```

```
$v = 121212;  
if (is_string($v)) {  
    echo "<br> $v es string";  
} else {  
    echo "<br> $v no es string";  
}
```

```
$v = "123.44";  
if (is_string($v)) {  
    echo "<br> $v es string";  
} else {  
    echo "<br> $v no es string";  
}
```

- [is_object\(\)](#) - Comprueba si una variable es un objeto

<https://www.php.net/manual/es/function.is-object.php>

```
$obj = new stdClass();  
$obj->students = array('Juan', 'Rosa', 'Ana');  
if (is_object($obj))  
    echo "<br> es object";  
else  
    echo "<br> NO es object";
```

- [is_array\(\)](#) - Comprueba si una variable es un array

<https://www.php.net/manual/es/function.is-array.php>

```
$alum = array('Juan', 'Rosa', 'Ana');  
if (is_array($alum))  
    echo "<br> es Array";  
else  
    echo "<br> NO es Array";
```

Para ver el contenido de arrays y objetos, se utiliza **var_dump()** o **print_r()**.

4.4.- Fechas

En PHP se puede trabajar con fechas al igual que cualquier otro lenguaje de programación. Esto es muy útil para tener un control en nuestra aplicación web del tiempo.

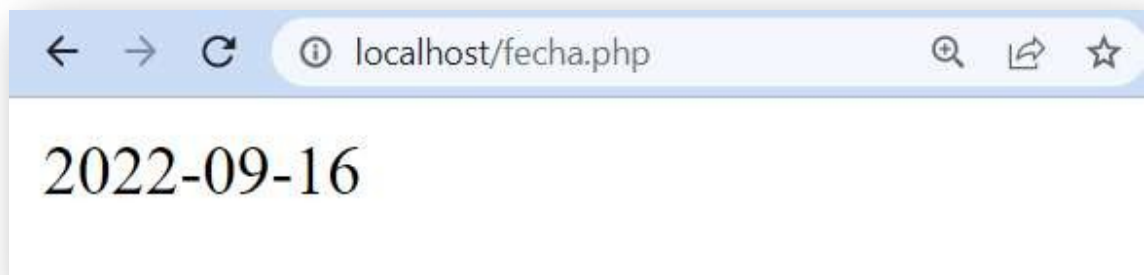
- **date()**

La función **date()** nos permite formatear una fecha, ya sea con el día, mes, año, horas, minutos, segundos, día de la semana, etc. Ésta recibirá dos

parámetros, el primero será justamente el formato de la fecha que queremos mostrar, y un segundo parámetro opcional que es el **timestamp** (ya veremos lo qué es eso)

Por ejemplo, si quisiéramos devolver el año, mes y día:

```
1 <?php
2     $fecha = date('Y-m-d');
3     echo $fecha;
4
5
```



Esto nos devolverá por pantalla la fecha en formato en inglés año-mes-día.

'Y' hace referencia al año (Year en inglés), 'm' al mes (month) y 'd' al día (day) Pero también podemos formatear el mismo en español:

```
<?php
    $fecha = date('d-m-Y');
    echo $fecha;
?>
```

Entonces el resultado será día-mes-año.



Si se pone en mayúscula el día y el mes sale el nombre:

También podemos recuperar la fecha y la hora **Hora (H) , minutos (i) y segundos (s):**

```
<?php
    $fecha_hora = date('d-m-Y H:i:s');
    echo $fecha_hora;
?>
```



- **mktime()**

Otra forma de recuperar la fecha y la hora en PHP es en formato **timestamp** (o marca de tiempo Unix), esto es la cantidad de segundos que va desde de la época Unix (1 de Enero de 1970 a las 00:00:00) hasta el momento en que se llama.

Esta función recibirá 6 parámetros: la hora, los minutos, los segundos, el mes, el día y el año (en ese orden):

```
<?php
    $mktime = mktime('19', '50', '02', '09', '07', '2020');
    echo $mktime;
?>
```

Esto nos devolverá por pantalla el valor: 1599501002

Lo que corresponde al 7 de septiembre del 2020, a las 19:50:02.

Por ejemplo, si quisiéramos recuperar la cantidad de segundos que hay desde esa fecha hasta el 7 de septiembre del 2020, a las 18:50:02:

```
<?php
    $mktime1 = mktime('19', '50', '02', '09', '07', '2020');
    $mktime2 = mktime('18', '50', '02', '09', '07', '2020');
    $resta = $mktime1 - $mktime2;
    echo $resta;
?>
```

Esto nos devolverá por pantalla: 3600

Que es la **diferencia de segundos** que hay desde una hora a la otra.

Ahora si quisiéramos devolver la fecha y la hora a través de un valor timestamp deberíamos hacer lo siguiente:

```
<?php
    $fecha = date('d-m-Y H:i:s', 1599501002);
    echo $fecha;
?>
```

Como se ve, esta vez la función **date()** recibirá un segundo parámetro que es el que le pasará la fecha que deberá formatear.

El resultado será: 07-09-2020 19:50:02

- **time()**

Otra función que nos permitirá devolver un valor de tipo timestamp es **time()**, en este caso recuperaremos el valor del momento en que la estamos llamando:

```
<?php
    $tiempo = time();
    $fecha = date('d-m-Y H:i:s');
    echo $tiempo;
    echo '<br>';
    echo $fecha;
```

```
?>
```

Esto nos devolverá el timestamp y el formato de la fecha, por ejemplo:

1594210589

08-07-2020 14:16:29

- **strtotime()**

Hasta ahora hemos pasado fechas timestamp en formato de fecha normal, pero si lo que queremos es hacer la inversa, deberíamos usar la función **strtotime()** que nos devolverá el valor timestamp:

```
<?php
    $fecha = date('10-09-2020 20:09:33');
    $timestamp = strtotime($fecha);
    echo $timestamp;
?>
```

Esto devolverá por pantalla: 1599761373

La función **date()** no sólo puede devolver la fecha o la hora, para ver los otros tipos de formato disponibles podemos consultar el manual de PHP:

<https://php.net/manual/es/function.date.php>

- **checkdate()**

Esta función nos mete en un terreno que aún no hemos entrado, las validaciones. Esta función valida que una fecha sea correcta, recibe tres parámetros: mes, día y año, en ese orden. Si la fecha es correcta devolverá **true**, sino **false**:

```
<?php
    $dia = 31;
    $mes = 11;
    $anio = 1986;

    if(checkdate($mes, $dia, $anio)){
        echo 'La fecha es correcta';
    }else{
        echo 'La fecha es incorrecta';
    }
?>
```

En este caso la función nos devolverá: La fecha es incorrecta

Porque obviamente 31 de noviembre de 1986 no es una fecha correcta, el mes de noviembre tiene hasta el día 30.



5.- Referencias bibliográficas

- ❖ Sanz, M. L. *Programación Web en el Entorno Servidor. (MF0492_3)*. Grupo Editorial RA-MA.
- ❖ Ganzábal, Xabier. *Desarrollo web en entorno servidor*. Editorial Síntesis.
- ❖ PHP (16 de septiembre de 22). *Manual oficial de PHP*
<https://www.php.net/manual/es/index.php>